



Informe Técnico: Implementación de Inventario Automatizado mediante Edge AI (TinyML)

Materia / Proyecto: Inteligencia Artificial Aplicada

Fecha: Junio 2026

1. Justificación del Uso de TinyML (Edge Computing)

El desafío principal del proyecto consistió en automatizar la supervisión y control del inventario en los laboratorios de cómputo de la institución. Las restricciones del entorno presentaban dos desafíos mayores:

- Falta de conexión estable:** Los laboratorios presentan interrupciones frecuentes de red Wi-Fi, lo cual imposibilita depender de un servidor en la Nube (API REST) para procesar las imágenes.
- Privacidad y Costos:** Enviar video en vivo o cientos de imágenes a un servidor externo genera latencia, consumo de ancho de banda y altos costos de computación.

Solución adoptada: Se implementó una solución **TinyML (Edge AI)** utilizando **TensorFlow Lite**. Esto permite compilar una Red Neuronal Convolucional (CNN) directamente dentro del smartphone del encargado, logrando inferencias en milisegundos sin utilizar conexión a internet, garantizando total disponibilidad (Modo Avión) y cero latencia de red.

2. Captura y Curaduría del Dataset

Uno de los mayores hallazgos durante el desarrollo fue el "sesgo de los datos". Inicialmente, el modelo fue entrenado con imágenes descargadas de internet, lo cual provocó un severo problema de generalización (todo el laboratorio era detectado erróneamente como "Útil de Escritorio").

Para solucionarlo, desarrollamos una herramienta propia nativa en Android llamada **CapturaDex**. Esta aplicación nos permitió extraer un dataset real:

- Entorno real:** Fotografías tomadas bajo la iluminación y con el mobiliario exacto del laboratorio de la institución.
- Metodología de Ráfaga Guiada:** La app CapturaDex toma múltiples imágenes con variaciones de ángulos de 600ms para robustecer la generalización espacial del modelo de IA.

3. Arquitectura del Modelo y Entrenamiento

Se empleó la técnica de **Transfer Learning** sobre la arquitectura **MobileNetV2** (pre-entrenada con millones de imágenes de ImageNet).

- Feature Extractor:** Se congelaron las capas base de MobileNetV2.
- Clasificador Final:** Se añadió una capa de reducción (*GlobalAveragePooling2D*), un Dropout del 20% para evitar sobreajuste, y una capa Densa de clasificación Softmax para 3 clases (*Herramienta Tecnológica, Útil de Escritorio, Mesa Vacía*).
- Preprocesamiento en Android:** Se implementó un algoritmo customizado de `ResizeWithCropOrPadOp` en Java para prevenir que la relación de aspecto de la cámara deforme los objetos, lo cual mejoró significativamente la precisión en el entorno móvil.

4. Cuantización y Cuadro Comparativo

Debido a que el modelo base trabaja con números decimales de alta precisión (Float32), resultaba muy pesado para la memoria caché del procesador móvil. Se aplicó una **Cuantización a Enteros de 8-Bits (INT8)**.

Característica	Modelo Base (Float32)	Modelo TinyML (INT8)	Beneficio
Tamaño en Disco	~14.00 MB	2.59 MB	Reducción del 81.5% en peso del APK
Requisito RAM	Alto	Muy Bajo	Evita cierres de la app por falta de memoria
Operaciones ALU	Punto Flotante	Enteros	Reduce el consumo de batería y el calentamiento
Latencia de Inferencia	~150 ms	< 30 ms	Permite escanear en tiempo real (30 FPS)

5. Arquitectura de Software y Diseño de Interfaz (UI/UX v2)

Para garantizar que el modelo matemático fuera útil para el usuario final, se construyó una aplicación nativa robusta con arquitectura reactiva:

- Gestión de Memoria y Base de Datos (Room SQLite):** Para evitar desbordamientos de memoria RAM al procesar listas gigantescas, se delegó la agrupación semántica al motor de base de datos a nivel de disco físico. Mediante consultas SQL `GROUP BY`, el celular es capaz de consolidar miles de registros en milisegundos sin latencia visual. El uso de **WAL (Write-Ahead Logging)** previene bloqueos de cámara durante la escritura.
- Debouncing y Retroalimentación Hardware:** La cámara inyecta 30 fotogramas por segundo, lo cual generaría falsos positivos. Para evitarlo, se implementó un mecanismo de *Debouncing* multihilo (protegido con cerrojos `synchronized`) que exige 10 fotogramas continuos de confirmación. Visualmente esto se apoya con una barra de progreso animada (Feedback instantáneo) y acústicamente con el generador de tonos del sistema de Android (`ToneGenerator`) que emite un bip de confirmación al registrar el objeto, solucionando el problema de la "Ceguera del Operador".
- UI Glassmorphism y Plan de Acción IA:** El diseño clásico fue reemplazado por componentes visuales *Premium*. Al finalizar el escaneo, en lugar de un cuadro de texto plano, la aplicación despliega dinámicamente un **Bottom Sheet Dialog** que expone el conteo categorizado e inyecta "Pasos Recomendados" (ej. reubicación o compra de equipos faltantes).

6. Conclusiones

La integración de **Edge Computing** demostró ser completamente superior al paradigma de computación en la nube para aplicaciones de inventario en tiempo real. Al procesar las imágenes en la fuente (el celular), hemos conseguido una aplicación robusta, veloz y completamente inmune a fallos de internet. La creación de un dataset curado localmente y el envoltorio arquitectónico de Android (Base de Datos Reactiva, Feedback Háptico y UI Empresarial) transformaron un modelo matemático en un producto final de alta utilidad para la institución.